Code No: 22PC0AI01 **R22**

GURU NANAK INSTITUTE OF TECHNOLOGY HYDERABAD

B. TECH III YEAR I SEMESTER, JULY-2026

ARTIFICIAL INTELLIGENCE (AI)

ASSIGNMENT-I

(Common to CSE)

Last date: 25.07.2026 (Saturday) **Max. Marks: 05**

QNo	Short Answer Questions	Bloom's Level
1.	Define Artificial Intelligence.	L1
2.	What do you mean by Weak AI?	L1
3.	What is meant by Perception?	L1
4	Expand PEAS.	L1
5	Define a heuristic.	L1

QNo	Long Answer Questions	Bloom's Level
1.	Explain Goal-Based Agents with neat sketch.	L2
2.	Illustrate the working of vacuum cleaner world example.	L2
3.	Discuss briefly uninformed search.	L2
4	Differentiate between BFS and DFS.	L2
5	Describe A* Search algorithm.	L2

***

INSTRUCTIONS:

1. Write answers in the given order only.
2. Write in your own handwriting. Your assignment is compared with your AI notes.
3. This assignment needs to be uploaded in google classroom, so **scan** before the submission.
4. Submit the assignment & get evaluated as soon as once you completed the assignment i.e., submit immediately after completion of your assignment-don't wait till last date.
5. Needs to submit the hardcopies on or before the last day without fail. If you absent on the last day of submission i.e., on 25th July, 2026 (Saturday) means you need to submit one day before (on Friday itself). You are not allowed to submit after the due date.
6. Failing the above rules will result in zero marks. So, follow the rules strictly and write correctly.

oOo

ARTIFICIAL INTELLIGENCE (AI) UNIT-I ASSIGNMENT

SHORT ANSWERS

1. Define Artificial Intelligence.

Ans: **Artificial Intelligence** (AI) is the simulation of human intelligence by machines to perform tasks like reasoning, learning, and decision-making.

2. What do you mean by Weak AI?

Ans: Weak AI is also known as specialized AI or narrow AI, which is artificial intelligence designed to perform one specialized task at a time, without real understanding or general reasoning ability.

3. What is meant by Perception?

Ans: Perception: Perception is the process by which an intelligent agent **receives, interprets, and understands sensory information** from the environment using its sensors.

4. Expand PEAS.

Ans: PEAS stands for:

P – Performance Measure

E – Environment

A – Actuators

S – Sensors

5. Define a heuristic.

Ans: Heuristic: A **heuristic** is a practical approach or rule-of-thumb that is used to make problem-solving faster and more efficient when exact methods are slow or infeasible.

LONG ANSWERS

1. Explain Goal-Based Agents with neat sketch.

Ans: Goal-based agents:

- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do. The agent needs to know its ***goal*** which describes desirable situations.
- **Goal-based agents** expand the capabilities of the model-based agent by having the "**goal**" information.
- They choose an action, so that they can achieve the goal.

- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenario are called searching and planning, which makes an agent proactive.

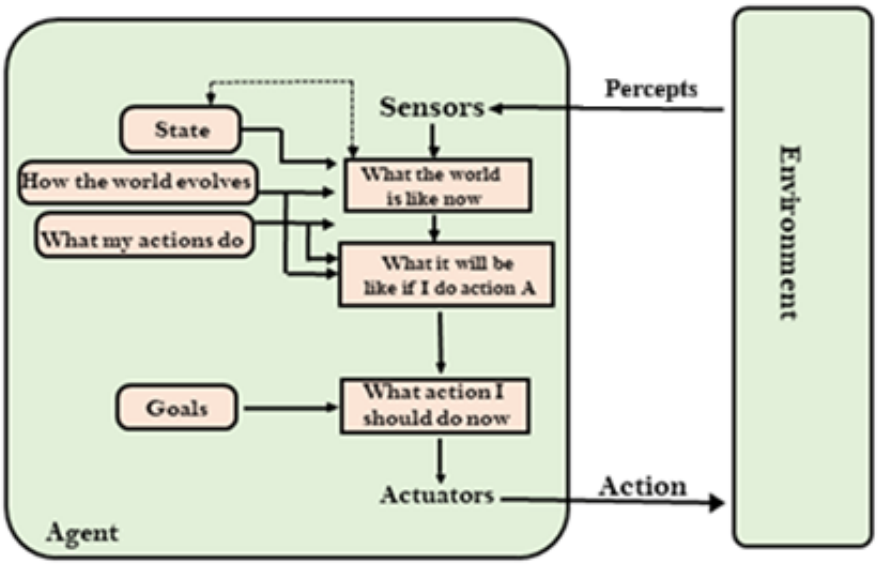

Figure: Goal-Based Agent and its Environment

- In the above figure, the agent receives **percepts** from the environment through its **sensors**, which tell it what the world is like now. It uses an internal **state** to understand how the world evolves and how its own actions affect the world.
- Based on this knowledge, the agent predicts **what the world will be like if it performs a certain action**. It then compares these predicted outcomes with its **goals** and chooses the best action that will help achieve those goals. Finally, the selected action is executed through the **actuators**, affecting the environment.

---

2. Illustrate the working of vacuum cleaner world example.

Ans: A vacuum cleaner can be considered as a simple agent, that makes decisions based solely on the current state of the environment, without considering the history of past states.

- It especially, operates based on basic sensor input and a set of predefined rules.

1. Description:

- The world has two squares (rooms), **A** and **B**, which may contain dirt.
- The agent (vacuum cleaner) can:
  - **Perceive** its location (A or B) and the check the cleanliness status (dirty or clean).
  - **Act** by moving left, right, sucking dirt, or do nothing.

2. Agent's Functionality:

- The **percept sequence** records everything what the agent perceives.

- The **agent function** determines the action based on the percept sequence.
- For example:
  - If the current square is dirty, the agent sucks the dirt.
  - If the current square is clean, the agent moves to the other square.

3. Agent Program:

- A simple agent program implements the above agent function.

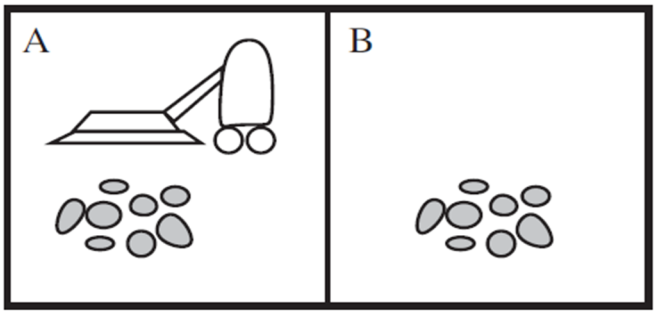

Figure 1.2: A vacuum-cleaner world with two locations

- If the current location is dirty, the vacuum cleaner performs the "suck" action to clean the area. If the current location is clean, the vacuum cleaner moves to the next location.

4. Actions in Different Scenarios:

- In this classical vacuum cleaner problem, we have two rooms and one vacuum cleaner. There is dirt in both the rooms and it is to be cleaned.
- The vacuum cleaner is present in any one of these rooms. So, we have to reach a state in which both the rooms are clean and are dust free.
- So, the following are the possible states in our vacuum cleaner problem. These can be well illustrated with the help of the following diagrams:

Percept sequence	Action
$[A, Clean]$	*Right*
$[A, Dirty]$	*Suck*
$[B, Clean]$	*Left*
$[B, Dirty]$	*Suck*
$[A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Dirty]$	*Suck*
⋮	⋮
$[A, Clean], [A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Clean], [A, Dirty]$	*Suck*

Figure 1.3: Partial tabulation of a simple agent function for the vacuum-cleaner world for above figure 1.2.

3. Discuss briefly uninformed search.

Ans:

Uninformed Search:

- **Uninformed Search** is also called **blind search**, which uses no information about the problem to guide the search.
- That means, these are the strategies have no additional information about states beyond that provided in the problem definition. All they can do is generate successors and distinguish a goal state from a non-goal state.
- The uninformed search does not contain any domain knowledge such as closeness, the location of the goal.
- Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called **blind search**. It examines each node of the tree until it achieves the goal node.

- Following are the various types of uninformed search algorithms:
  1. Breadth-first search
  2. Depth-first search
  3. Uniform cost search n
  4. Iterative deepening Depth-first search
  5. Bidirectional search
  6. Depth-limited Search

1. Breadth-first search (BFS):

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadth wise in a tree or graph, so it is called **breadth-first search**.

- BFS algorithm starts searching from the root node of the tree and expands all ***successor*** nodes at the current level before moving to nodes of next level.

2. Depth-first Search (DFS):

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.

- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.

3. Uniform cost search:

- Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph.

- The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost.

4. Iterative deepening Depth-first search:

- Iterative deepening search (or iterative deepening depth-first search) is a general strategy, is a combination of DFS and BFS algorithms.
- This search algorithm finds out the best depth limit. It does this by gradually increasing the limit—first 0, then 1, then 2, and so on until a goal is found.

5. Bidirectional Search:

- The idea behind bidirectional search is to run two simultaneous searches—one forward from the initial state and the other backward from the goal—hoping that the two searches meet in the middle.

6. Depth-Limited Search Algorithm :

- A **depth-limited search** algorithm is similar to depth-first search with a predetermined limit.

- Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

---

4. Differentiate between BFS and DFS.

Ans:

BFS	DFS
BFS stands for Breadth First Search.	DFS stands for Depth First Search.
BFS(Breadth First Search) uses Queue data structure for finding the shortest path.	DFS(Depth First Search) uses Stack data structure.
BFS can be used to find single source shortest path in an unweighted graph, because in BFS, we reach a vertex with minimum number of edges from a source vertex.	In DFS, we might traverse through more edges to reach a destination vertex from a source.
BFS is more suitable for searching vertices which are closer to the given source.	DFS is more suitable when there are solutions away from source.
BFS considers all neighbors first and therefore not suitable for decision making trees used in games or puzzles.	DFS is more suitable for game or puzzle problems. We make a decision, then explore all paths through this decision. And if this decision leads to win situation, we stop.
The Time complexity of BFS is O(V + E) when Adjacency List is used and O(V^2) when Adjacency Matrix is used, where V stands for vertices and E stands for edges.	The Time complexity of DFS is also O(V + E) when Adjacency List is used and O(V^2) when Adjacency Matrix is used, where V stands for vertices and E stands for edges.

5. Describe A* Search algorithm.

Ans: A* Search:

- The A* Search algorithm is used to find the optimal path from the initial state to goal state.
- In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence, we can combine both costs as follows, and this sum is called as a **fitness number**.
- A* Search algorithm evaluates nodes by using the function,

$$f(n) = g(n) + h(n) \quad \longrightarrow \quad (1)$$

where,
f(n) is the estimated cost of the cheapest solution through ***n***

g(n) is the actual cost of reaching node ***n*** from the current node

h(n) is the heuristic estimate of the cost to reach the goal from node ***n***.

- **A* algorithm** is similar to Uniform-Cost Search (UCS) except that it uses ***g(n)*** + ***h(n)*** instead of ***g(n)***.

A* Search Algorithm:

Step 1: Place the start node in the priority queue: OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stop.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (***g*** + ***h***), if node ***n*** is a goal node, then return success and stop.

Step 4: If node ***n*** is not a goal node, then expand the node ***n*** and generate all of its successors, and put ***n*** into the closed list.
For each **successor n'**, check whether **n'** is already in the OPEN or CLOSED list,
⇒ if not, then compute evaluation function for **n'** and place into OPEN list.
⇒ If node **n'** is already in OPEN or CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

Step 5: Return to **Step 2**.

- **A* search** algorithm finds the shortest path through the search-space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster.

**** THE END ****